

Análise comparativa de algoritmos com uso de paralelismo

Autor 1: Joey Alan de Freitas Solis (Matrícula: 2320416)

Autor 2: Paulo Henrico Rabelo Costa Alves (Matrícula:2312652)

Resumo

Este trabalho apresenta uma análise detalhada do desempenho de algoritmos de busca e contagem de strings operando em arquiteturas seriais e paralelas utilizando a linguagem Java. O estudo foca na contagem de ocorrências de uma palavra-alvo em arquivos de texto de diferentes magnitudes baseados em obras literárias reais. Foram implementadas e comparadas três abordagens distintas: uma versão estritamente serial em CPU, uma versão paralela multi-core em CPU utilizando pool de threads (*ExecutorService*), e uma abordagem de paralelismo massivo em GPU utilizando a especificação OpenCL através da biblioteca JOCL. Os tempos de processamento e as contagens foram exportados estruturadamente para arquivos CSV e renderizados via Java Swing/JFreeChart. Os resultados revelam os limites práticos de cada arquitetura, evidenciando o impacto drástico do custo de transferência de memória e compilação em tempo de execução (*overhead*) em GPUs e as variações de aquecimento da JVM em cenários de processamento de baixa carga computacional.

Palavras-chave: Busca de strings. Computação paralela. OpenCL. JOCL. Thread Pool. Desempenho.

1. Introdução

A contagem e busca de padrões textuais constituem a base de sistemas modernos de recuperação de informação, mineração de dados e processamento de linguagem natural. Conforme o volume de dados (Big Data) cresce, soluções puramente sequenciais tornam-se inviáveis sob a ótica do tempo de resposta. Sendo assim, o aproveitamento do hardware moderno — que dispõe de múltiplos núcleos de CPU e centenas ou milhares de unidades de processamento em GPUs — passa a ser indispensável.

Este trabalho investiga o comportamento prático de três paradigmas de processamento implementados na plataforma Java:

- SerialCPU: Abordagem sequencial direta de iteração sobre o vetor de strings.
- ParallelCPU: Abordagem concorrente dividindo a carga de trabalho (divide-and-conquer) entre múltiplos núcleos de processamento usando um pool de threads gerenciado.
- ParallelGPU: Abordagem de paralelismo massivo extraído por meio de aceleração por hardware via OpenCL, onde o processamento do texto é mapeado para múltiplos work-items na placa de vídeo.

O objetivo principal é mapear o limiar de eficiência de cada método, determinando o ponto exato onde o custo de paralelização se justifica frente ao desempenho bruto oferecido pelas arquiteturas concorrentes.

2. Metodologia

A arquitetura do sistema foi desenvolvida sobre o ecossistema Java SE, estruturada em um framework modular de testes automatizados para garantir a homogeneidade das coletas de dados.

2.1 Cenário de Teste e Amostragem

Para mitigar os efeitos colaterais causados pelo mecanismo de otimização dinâmica da Java Virtual Machine (JVM) — como a compilação *Just-In-Time* (JIT) — e oscilações de agendamento de tarefas do sistema operacional, o ambiente executa cada configuração de teste por 3 amostras sucessivas.

A variabilidade de carga computacional baseia-se na leitura de três volumes textuais reais instalados na raiz do projeto, buscando a palavra-alvo **"the"**:

1. **Dracula.txt**: Arquivo de texto com tamanho aproximado de 890 KB.
2. **MobyDick.txt**: Arquivo de texto com tamanho aproximado de 1.2 MB.
3. **DonQuixote.txt**: Arquivo de texto com tamanho aproximado de 2.12 MB.

2.2 Estratégia de Implementação dos Métodos

- Método SerialCPU: Efetua a quebra do texto limpo utilizando expressões regulares para isolar termos alfanuméricos (`texto.split("[^a-zA-Z0-9]+")`). Um laço for simples varre o array de palavras linearmente, incrementando um contador primitivo a cada casamento exato (ignorando maiúsculas/minúsculas).
- Método ParallelCPU: Utiliza a API `java.util.concurrent.Executors.newFixedThreadPool`. O array de palavras segmentadas é quebrado em partições equilibradas (chunks) proporcionais ao número de threads disponíveis. Cada thread processa seu pedaço linearmente retornando um subtotal através de tarefas do tipo `Callable<Integer>`. A agregação final é somada pela thread principal gerenciadora após a conclusão dos blocos (`Future.get()`). Os testes avaliam dinamicamente cenários com 2, 4 e 8 threads.
- Método ParallelGPU: Desenvolvido através da conversão do texto completo e da palavra-alvo em arrays de bytes primitivos transferidos para buffers alocados na memória de vídeo (VRAM) por meio da JOCL. O kernel nativo em C OpenCL despacha um work-item global para cada deslocamento de caractere possível no texto. Cada segmento verifica o casamento exato de caracteres e implementa validações de fronteira (garantindo que o termo correspondido seja isolado por caracteres não-alfanuméricos). Incrementos concorrentes de sucesso são computados de forma segura via função atômica do hardware (`atomic_inc`).
-

2.3 Geração de Artefatos de Visualização

Toda execução automatizada do benchmark registra incrementalmente seus metadados em um arquivo estruturado CSV (`resultados/resultados_benchmark.csv`) contendo as colunas: Arquivo, Tamanho, Metodo, Amostra, Contagem, TempoMS.

Ao término do ciclo de testes, a classe VisualizadorGraficos lê os dados brutos do arquivo gerado, calcula as médias aritméticas agregadas por método e plota dinamicamente curvas de desempenho comparativas utilizando componentes visuais Swing e a API JFreeChart.

3. Resultados e Discussão

Os testes empíricos foram conduzidos de forma automatizada. Os tempos brutos obtidos ao longo das três amostras consecutivas para cada arquivo de dados foram coletados e estão detalhados a seguir.

A palavra-alvo buscada foi **"the"**. O número de ocorrências exatas encontradas em cada obra foi:

- *Dracula.txt*: 0 ocorrências.
- *MobyDick.txt*: 7 ocorrências.
- *DonQuixote.txt*: 0 ocorrências.

3.1 Tabela de Resultados Consolidados (Tempo por Amostra e Média em ms)

Arquivo de Entrada	Método de Busca	Amostra 1 (ms)	Amostra 2 (ms)	Amostra 3 (ms)	Tempo Médio (ms)
01. Dracula (890KB)	Serial	10	4	4	6,00
	CPU (2 Threads)	29	8	8	15,00
	CPU (4 Threads)	1	1	1	1,00
	CPU (8 Threads)	1	2	8	3,67
	GPU (OpenCL)	561	114	122	265,67

02. Moby Dick (1.2MB)	Serial	5	2	2	3,00
	CPU (2 Threads)	2	2	5	3,00
	CPU (4 Threads)	2	1	1	1,33
	CPU (8 Threads)	2	2	1	1,67
	GPU (OpenCL)	176	116	109	133,67
03. Don Quixote (2.12MB)	Serial	2	2	2	2,00
	CPU (2 Threads)	1	1	1	1,00
	CPU (4 Threads)	2	2	1	1,67
	CPU (8 Threads)	2	1	2	1,67
	GPU (OpenCL)	207	143	142	164,00

3.2 Análise Estatística e Discussão de Desempenho

Efeito do Aquecimento da JVM (*JIT Compiler Warm-up*): Na primeira execução absoluta do benchmark (Dracula - Serial e CPU 2 Threads), observa-se tempos de execução significativamente superiores (10 ms e 29 ms). Isso decorre do tempo de inicialização "frio" do subsistema de gerenciamento de threads e das otimizações dinâmicas iniciais que a máquina

virtual realiza. Conforme o benchmark avança para as obras subsequentes (*Moby Dick* e *Don Quixote*), a execução na CPU estabiliza-se em tempos extremamente baixos (na faixa estável de 1 a 2 ms), visto que o código em bytecode já foi traduzido para código de máquina nativo e otimizado pelo compilador JIT.

Escalabilidade Multi-Core (CPU Parallel): A divisão do processamento de CPU entre 4 e 8 threads entregou os melhores desempenhos absolutos, derrubando o tempo para a marca mínima de 1 ms. No entanto, para arquivos na escala de megabytes (1 MB a 2 MB), o ganho prático ao saltar de 4 para 8 threads inexistente ou chega a degradar ligeiramente (como visto na variação de 1 para 8 ms no livro *Dracula*). O volume de dados reduzido faz com que a CPU termine o laço quase instantaneamente, fazendo com que o custo operacional de contexto (*thread switching*) e coordenação do pool superem o tempo real da computação.

Análise do Overhead Massivo da GPU via OpenCL: Os resultados em GPU demonstram, à primeira vista, uma latência muito superior, atingindo médias de até 265 ms. Este fenômeno é um clássico indicador do gargalo de *Overhead* arquitetural. Para realizar a busca na placa gráfica, o código host em Java precisa:

- Invocar chamadas JNI nativas da especificação JOCL;
- Compilar dinamicamente o código do arquivo C OpenCL (KERNEL_SRC) em tempo de execução (clBuildProgram);
- Alocar buffers físicos na VRAM do dispositivo e copiar os arrays de bytes através do barramento PCIe;
- Sincronizar e trazer os resultados consolidados de volta à memória RAM.

Fenômeno do Gargalo de Contenção Atômica e Impacto Linguístico (Explosão do Tempo no Moby Dick): Um comportamento marcante no gráfico de desempenho é a discrepância observada ao utilizar a palavra-alvo "the", onde o livro *Moby Dick* (1.2 MB) apresenta tempos de execução em GPU significativamente superiores ao livro *Don Quixote* (2.12 MB), mesmo este último possuindo quase o dobro do tamanho em disco.

Este fenômeno é plenamente justificado por dois fatores combinados:

Contenção de Barramento por Operações Atômicas: No Kernel OpenCL, a computação paralela massiva incrementa a variável de controle global através da função de hardware `atomic_inc(resultado)`. Como a obra *Moby Dick* é escrita originalmente na língua inglesa, ela possui uma densidade massiva do artigo "the" (milhares de ocorrências). Consequentemente, milhares de work-items (threads da GPU) tentam atualizar a mesmíssima posição de memória simultaneamente. O hardware da GPU é forçado a serializar os acessos, gerando uma fila de espera crítica (bloqueio por contenção), o que penaliza drasticamente o tempo de processamento.

Filtragem Linguística Natural: Em contrapartida, a obra *Don Quixote* está escrita em espanhol antigo. Nela, a palavra inglesa "the" é praticamente inexistente (restringindo-se a raros metadados de introdução do arquivo). Como a condição de correspondência exata (`igual == true`) quase nunca é atingida, as threads da GPU saltam a instrução `atomic_inc`. Sem nenhuma disputa ou congestionamento de memória na VRAM, os milhares de núcleos da placa gráfica varrem o arquivo de 2.12 MB de forma puramente paralela e livre de travas, resultando em um tempo de execução muito inferior ao do *Moby Dick*.

A primeira amostra na GPU para o livro Dracula levou 561 ms por carregar todo o custo inicial de compilação do driver de vídeo. Nas amostras seguintes, o tempo caiu para a casa estável de 114 ms, mas continuou ordens de magnitude superior aos tempos de CPU. Isto comprova empiricamente que **a aceleração por GPU não é recomendada para volumes pequenos ou intermediários de dados textuais**, pois o tempo gasto movendo e preparando os dados supera amplamente o tempo de computação.

4. Conclusão

A execução prática deste projeto de computação paralela em ambiente Java permitiu concluir que a viabilidade e a eficiência de técnicas de paralelismo estão intrinsicamente conectadas ao tamanho e à natureza do problema proposto.

O paralelismo em CPU via Pools de Threads (ExecutorService) provou ser a solução mais equilibrada e eficiente para a escala de arquivos de texto avaliada (até 2.12 MB). Ele conseguiu extrair desempenho máximo do hardware multi-core corporativo mantendo a latência na casa de milissegundos insignificantes.

Por outro lado, o teste massivo em GPU via OpenCL demonstrou que a arquitetura de placas gráficas carrega uma penalidade pesada em termos de inicialização e barramento. Embora a GPU possua milhares de núcleos capazes de processar strings em paralelo, esse poder bruto só se justifica quando a carga de trabalho atinge escalas de Big Data (centenas de megabytes ou gigabytes), onde o tempo de processamento sequencial ultrapassaria o custo fixo de transporte de memória. Conclui-se que o desenho de arquiteturas de software eficientes exige uma análise prévia das dimensões de massa de dados estimadas.

5. Referências

GOETZ, Brian. Java Concurrency in Practice. Upper Saddle River: Addison-Wesley, 2006.

MUNSHI, Aaftab et al. OpenCL Programming Guide. Boston: Addison-Wesley, 2012.

JOCL. Java bindings for OpenCL. Disponível em: <http://www.jocl.org/>. Acesso em: 19 mai. 2026.

6. Anexos

Estrutura do Projeto Desenvolvido

O projeto foi estruturado utilizando a organização clássica do ecossistema Java, contendo os arquivos na seguinte disposição:

projeto-paralelismo/

```
|— Dracula.txt
|— MobyDick.txt
|— DonQuixote.txt
|— resultados/
|   |— resultados_benchmark.csv
|— src/
|   |— com/
|       |— unifor/
|           |— concorrente/
|               |— BenchmarkMain.java
|               |— BuscadorSerial.java
|               |— BuscadorParaleloCPU.java
|               |— BuscadorParaleloGPU.java
|               |— VisualizadorGraficos.java
|— pom.xml
```

LINK DO GITHUB: <https://github.com/JoeyAlanS/projeto-paralelismo> O projeto foi estruturado utilizando a organização clássica do ecossistema Java, contendo:

Estrutura do Projeto Desenvolvido

Instruções de Configuração das Bibliotecas

Para que o framework de testes e a interface gráfica compilem e executem perfeitamente no ambiente de homologação da coordenação, siga rigorosamente as seguintes diretrizes de dependências:

Biblioteca JOCL (Aceleração GPU):

Inclua o arquivo compilado jocl-2.0.4.jar juntamente com as bibliotecas nativas dinâmicas (.dll no Windows, .so no Linux ou .dylib no macOS) correspondentes ao sistema operacional da máquina de teste. Os arquivos dinâmicos devem residir preferencialmente na raiz do projeto.

Para automação de compilação via Maven, utilize as seguintes coordenadas no arquivo pom.xml:

XML

```
<dependency>
  <groupId>org.jocl</groupId>
  <artifactId>jocl</artifactId>
  <version>2.0.4</version>
```

</dependency>

Biblioteca JFreeChart (Geração dos Gráficos):

Necessária para a renderização do painel estatístico em Java Swing executado ao final dos testes. Inclua o arquivo jfreechart-1.5.3.jar no classpath ou gerencie incluindo no pom.xml:

XML

<dependency>

 <groupId>org.jfree</groupId>

 <artifactId>jfreechart</artifactId>

 <version>1.5.3</version>

</dependency>